BIT4 — PORT MAP

Hardware I/O Reference | Last updated: 27.06.2026

STORE ports (assembly → VPU/system)

Addr	Command	Description	Group
0x00	Draw mode	0 = normal (PSET) • 1 = XOR (draw/erase)	VPU
0x01	Backspace	Cursor 1 char back + erase that cell	VPU
0x02	Cursor LEFT	Cursor 1 char back, no erase (left arrow)	VPU
0x03	X HIGH	Cursor X high nibble (value × 16 × scale)	VPU
0x04	Y absolute	Cursor Y (value × scale × 8 — full screen)	VPU
0x05	RAM bank switch	Switch active RAM bank (0–15)	SYS
0x06	Draw sprite	Draw 4×5 sprite from bank 15 (rows 0–4)	VPU
0x07	X absolute	Cursor X low nibble (value × scale)	VPU
0x08	Cursor visibility	0 = hide, ≠0 = show (render-side cursor)	VPU
0x09	ROM page switch	Select character ROM page for PRINT	VPU
0x0A	CLS	Clear screen, reset cursor to (0,0)	VPU
0x0B	Newline	Cursor Y += value × scale (relative)	VPU
0x0C	Fast-forward X	Cursor X += value × scale	VPU
0x0D	Carriage return	Cursor X = 0	VPU
0x0E	Color set	Active color code (0–15, ZX-style attr)	VPU
0x0F	Draw 4 pixels	Draw 4 raw pixels at cursor, advance	VPU

LOAD ports (VPU/system → assembly stack)

Addr	Source	Description	Group
0x00	(free)	—	—
0x01	Random	4-bit LFSR random value (period 15)	SYS
0x02	Cursor X read	Current cursor X / scale (CursorX/CharAdvX)	VPU
0x03	(free)	—	—
0x04	(free)	—	—
0x05	(free)	—	—
0x06	Keyboard page	Last keypress ROM page (0–3) or 0	KBD
0x07	(free)	—	—
0x08	Keyboard code	Last keypress char/scancode ID	KBD
0x09	(free)	—	—
0x0A	(free)	—	—
0x0B	(free)	—	—
0x0C	(free)	—	—
0x0D	(free)	—	—

Addr	Source	Description	Group
0x0E	(free)	—	—
0x0F	(free)	—	—

Sprite engine (bank 15)

A 4×5 sprite lives in bank 15, rows 0–4 (one row per address, 4 bits per row). **store 0006** draws it at the current cursor position. The current color (**store 000E**) and draw mode (**store 0000**) apply. With XOR mode on, drawing the same sprite twice at the same place erases it — this is how the animation "erase old, draw new" loop works without a CLS.

Step	Assembly fragment	Effect

1. Load sprite	`mov ax,0110 → storeb 15,0000`
	`mov ax,1111 → storeb 15,0001`
	…
		Fill bank 15 rows 0–4 with bit patterns

2. Enable XOR	`mov ax,0001 → store 0000`	Draw mode = XOR
3. Position	`mov ax,0101 → store 0007  (X)` `mov ax,0011 → store 0004  (Y)`	Cursor → (X,Y)
4. Draw	`mov ax,0000 → store 0006`	Sprite painted at cursor (XOR)
5. Erase same	`store 0006 (again, same X,Y)`	XOR cancels → sprite removed

Display configuration (32×32 downscale)

The screen was downscaled from 128×128 logical to **32×32 logical** to better fit the 4-bit ISA. Physical pixels in the host window stayed at 256×256 via a larger pixel scale. This brought the Y range within reach of a single 4-bit value (× scale × 8) without latch tricks, gave sprites a meaningful screen footprint, and aligns the machine with handheld-class targets (Game & Watch family, simple Pong/Breakout, etc.).

Constant	Value	Notes
Width / Height (px)	256 × 256	Host window framebuffer
PixelScale	8	Each logical px = 8 host px
Logical area	32 × 32	What programs address
BlockSize (attr cell)	8	ZX-style color cell
BlocksPerRow / Col	32 / 32	Width / BlockSize
Attr count	1024	BlocksPerRow × BlocksPerCol
Char glyph	4 × 5	ROM glyph size (unchanged)
Char advance	5 × 6	X / Y step per print

Changelog since 19.06.2026

- **store 0000** added — draw mode register (PSET / XOR), GW-BASIC style.
- **store 0002** repurposed — was cursor block draw, now cursor LEFT (no erase).
- **store 0004** multiplier increased from ×scale to ×scale×8 — Y now reaches full screen with a single 4-bit value.
- **store 0006** added — sprite draw from bank 15.
- **store 0008** added — cursor visibility toggle. Cursor is now drawn render-side, never written to the buffer (no trail).
- **load 0001** added — 4-bit LFSR random number.
- Bank 15 reserved as sprite data bank (rows 0–4 = one 4×5 sprite).
- Display downscaled to 32×32 logical (PixelScale 8).